

Projection aléatoire : embeddings efficaces pour graphes hétérogènes

Présentation technique de 30 minutes

Your Name

Your Institution

July 1, 2025

Plan de la présentation (30 minutes)

- 1 Définition du problème et motivation
- 2 Theoretical Foundation
- 3 FastRP Algorithm & Implementation
- 4 Optimization & Learning
- 5 Optimisation des paramètres
- 6 Benefits vs. Random Walk Methods
- 7 Results & Performance
- 8 Key Takeaways

Notre jeu de données

- 28,702 Auteurs
- 20 Conférences
- 8,920 Termes
- Types de relations multiples : Auteur-Auteur, Auteur-Conférence, Auteur-Terme

Objectif

Apprendre des embeddings de dimension 256 qui capturent les relations sémantiques afin de prédire les collaborations

Pourquoi est-ce difficile ?

- Graphe hétérogène (plusieurs types de nœuds)
- Connectivité clairsemée
- Échelle : gérer efficacement plus de 28 000 nœuds
- Complexité sémantique : chaque type de relation a un sens différent

Algorithme de marche aléatoire :

- Pour chaque nœud :
 - Générer des marches aléatoires : $A \rightarrow C \rightarrow A \rightarrow T \rightarrow A \rightarrow \dots$
 - Considérer les marches comme des phrases
 - Appliquer Word2Vec (Skip-gram) pour apprendre les embeddings

Problèmes

- **Coût computationnel** : millions de marches nécessaires
- **Mémoire** : stockage de toutes les séquences
- **Biais d'échantillonnage** : les marches ratent la structure globale
- **Sensibilité aux hyperparamètres** : longueur et nombre de marches, etc.

Exemple : Pour 28k auteurs, 10 marches $\times$ 100 pas = 28M pas !

Idée clé : éviter les marches

Instead of random walks, use random projections:

- 1 **Create sparse random matrix R** (cheap to compute)
- 2 **Apply graph operations directly: MR , M^2R , M^3R**
- 3 **Combine features intelligently** with learned weights

Pourquoi ça marche

Johnson-Lindenstrauss Lemma guarantees that random projections preserve distances in high-dimensional spaces.

Analogy

Instead of exploring a city by taking random walks, we take aerial photos from different angles and combine them!

Theorem (Johnson-Lindenstrauss Lemma (1984))

For any set of n points in high-dimensional space, there exists a mapping to $O(\log n)$ dimensions that preserves all pairwise distances within $(1 \pm \epsilon)$.

Formulation formelle

For points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^d$, there exists random matrix $\mathbf{R} \in \mathbb{R}^{d \times k}$ where $k = O(\log n / \epsilon^2)$ such that:

$$(1 - \epsilon) \|\mathbf{x}_i - \mathbf{x}_j\|^2 \leq \|\mathbf{R}^T \mathbf{x}_i - \mathbf{R}^T \mathbf{x}_j\|^2 \leq (1 + \epsilon) \|\mathbf{x}_i - \mathbf{x}_j\|^2$$

Conséquences pour nous

- Random projections preserve neighborhood structure
- We can work in lower dimensions without losing information
- Provides theoretical justification for our approach

Étape 1 : Construction de la matrice de projection aléatoire

Dimensions matricielles

$\mathbf{R} \in \mathbb{R}^{n \times d}$ (28,702 $\times$ 256 in our case)

Motif de sparsité

Each column has exactly $s = 3$ non-zero entries:

$$\mathbf{R}_{ij} = \begin{cases} \frac{+1}{\sqrt{s}} & \text{with probability } \frac{s}{n} \\ \frac{-1}{\sqrt{s}} & \text{with probability } \frac{s}{n} \\ 0 & \text{otherwise} \end{cases}$$

Example for 6 authors, 4 dimensions, $s = 2$

$$\mathbf{R} = \begin{bmatrix} 0 & +0.7 & 0 & -0.7 \\ -0.7 & 0 & +0.7 & 0 \\ 0 & 0 & -0.7 & +0.7 \\ +0.7 & -0.7 & 0 & 0 \\ 0 & +0.7 & -0.7 & 0 \\ -0.7 & 0 & 0 & +0.7 \end{bmatrix}$$

Étape 2 : Pondération par degré (paramètre α)

Problem

High-degree nodes (prolific authors) dominate embeddings

Solution

Degree weighting with parameter $\alpha = -0.5$:

$$\mathbf{R}' = \mathbf{D}^{\alpha} \mathbf{R}$$

Example with $\alpha = -0.5$

Author degrees: [100, 10, 5, 50, 20, 8]

$$\mathbf{D}^{(-0.5)} = \text{diag}([0.10, 0.32, 0.45, 0.14, 0.22, 0.35])$$

Before weighting:

A_1 (degree=100): [0, +0.7, 0, -0.7]

A_2 (degree=10): [-0.7, 0, +0.7, 0]

Étape 3 : Matrices d'adjacence des méta-chemins

Les méta-chemins capturent les relations sémantiques

$$\mathbf{ACA} \text{ (Author-Conference-Author): } \mathbf{M}_{ACA} = \mathbf{A}_{AC} \mathbf{A}_{CA} \quad (1)$$

$$\mathbf{ATA} \text{ (Author-Term-Author): } \mathbf{M}_{ATA} = \mathbf{A}_{AT} \mathbf{A}_{TA} \quad (2)$$

$$\mathbf{AAA} \text{ (Author-Author-Author): } \mathbf{M}_{AAA} = \mathbf{A}_{AA} \mathbf{A}_{AA} \quad (3)$$

Exemple de dimensions

- $\mathbf{A}_{AC}$: $28,702 \times 20$ (authors $\times$ conferences)
- $\mathbf{A}_{CA}$: $20 \times 28,702$ (conferences $\times$ authors)
- $\mathbf{M}_{ACA}$: $28,702 \times 28,702$ (authors $\times$ authors)

Interprétation

$\mathbf{M}_{ACA}[i, j]$ = number of conferences authors i and j both published in

Étape 4 : Normalisation (paramètre β)

Problem

Some authors have many connections, others few

Solution

Row normalization with $\beta = -1.0$:

$$\tilde{\mathbf{M}} = \mathbf{D}^{\beta} \mathbf{M}$$

Example

Raw $\mathbf{M}_{ACA}$ row sums: [50, 5, 100, 20] (conferences per author)

$$\mathbf{D}^{(-1)} = \text{diag}([1/50, 1/5, 1/100, 1/20])$$

Before normalization:

Author₁ : [10, 5, 30, 5] (shared conferences)

Author₂ : [2, 0, 1, 2]

Innovation clé

Au lieu de calculer explicitement $\mathbf{M}^2$, $\mathbf{M}^3$, on itère :

Algorithme des puissances de matrice (itératif) :

- $\mathbf{U}_0 = \mathbf{R}'$ (Initial : matrice aléatoire pondérée)
- $\mathbf{U}_1 = \text{normalize}(\tilde{\mathbf{M}}\mathbf{U}_0)$ (caractéristiques 1 saut)
- $\mathbf{U}_2 = \text{normalize}(\tilde{\mathbf{M}}\mathbf{U}_1)$ (2-hop features)
- $\mathbf{U}_3 = \text{normalize}(\tilde{\mathbf{M}}\mathbf{U}_2)$ (3-hop features)

Concrete Example (ACA meta-path)

$$\mathbf{U}_0 = \mathbf{R}' \in \mathbb{R}^{28,702 \times 256} \quad (\text{Random starting features})$$

$$\mathbf{U}_1 = \tilde{\mathbf{M}}_{ACA} \mathbf{U}_0 \quad (\text{Authors connected via 1 conference})$$

$$\mathbf{U}_2 = \tilde{\mathbf{M}}_{ACA} \mathbf{U}_1 \quad (\text{Authors connected via 2 conferences})$$

$$\mathbf{U}_3 = \tilde{\mathbf{M}}_{ACA} \mathbf{U}_2 \quad (\text{Authors connected via 3 conferences})$$

Memory Efficiency

Étape 6 : Collecte multi-échelle des caractéristiques

Pour chaque couple méta-chemin $\times$ puissance

Meta-paths: [AAA, ACA, ATA]

Powers: [1, 2, 3]

Total features: $3 \times 3 = 9$ feature matrices

Feature tensor: $\mathbf{F} \in \mathbb{R}^{9 \times 28,702 \times 256}$

Interprétation des caractéristiques

- $\mathbf{F}[0]$: AAA¹ - Direct collaborations
- $\mathbf{F}[1]$: AAA² - 2-hop collaborations
- $\mathbf{F}[2]$: AAA³ - 3-hop collaborations
- $\mathbf{F}[3]$: ACA¹ - Shared conferences
- $\mathbf{F}[4]$: ACA² - Conference communities
- $\mathbf{F}[5]$: ACA³ - Broader research areas

Étape 7 : Combinaison de caractéristiques apprenable

Problem

Which features are most important? Let the model decide!

Poids apprenables avec normalisation softmax

$$\theta \in \mathbb{R}^9 \quad (\text{Learnable parameters}) \quad (4)$$

$$\mathbf{w} = \text{softmax}(\theta) \quad \text{where} \quad \sum_{i=0}^8 w_i = 1 \quad (5)$$

(6)

$$\mathbf{Z} = \sum_{i=0}^8 w_i \times \mathbf{F}[i] \quad (\text{Final embedding}) \quad (7)$$

Example learned weights

AAA ¹ : 0.05	AAA ² : 0.12	AAA ³ : 0.08
ACA¹: 0.25	ACA²: 0.30	ACA³: 0.15

Étape 8 : Prédiction de lien par distance

Score basé sur la distance

For authors i and j , get embeddings $\mathbf{z}_i, \mathbf{z}_j \in \mathbb{R}^{256}$:

$$\text{distance}^2 = \|\mathbf{z}_i - \mathbf{z}_j\|^2 = \sum_{k=1}^{256} (z_{i,k} - z_{j,k})^2 \quad (8)$$

(9)

$$\text{probability} = \sigma(\gamma - \lambda \times \text{distance}^2) \quad (10)$$

Intuition

- Small distance $\rightarrow$ High probability of collaboration
- Large distance $\rightarrow$ Low probability of collaboration

Paramètres apprenables

- γ (intercept): Overall collaboration likelihood
- λ (slope): How much distance matters

Fonction de perte : deux termes

$$\mathcal{L} = \mathcal{L}_{BCE} + \lambda_{entropy} \times \mathcal{L}_{entropy}$$

1. Entropie croisée binaire (BCE)

$$\mathcal{L}_{BCE} = -[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})]$$

- Standard link prediction loss
- Weighted for class imbalance (3:1 negative:positive ratio)

2. Régularisation entropique

$$\mathcal{L}_{entropy} = - \sum_{i=1}^9 w_i \log(w_i)$$

- Prevents model from using only one feature type
- Encourages diverse feature usage

Principe

- ① Les matrices de caractéristiques $\mathbf{F}_f$ sont pré-calculées pour chaque méta-chemin et chaque puissance.
- ② Les poids w_f (issus d'un softmax sur les paramètres θ_f) combinent ces matrices : $\mathbf{E} = \sum_f w_f \mathbf{F}_f$.
- ③ Pour chaque paire d'auteurs i, j : score $s_{ij} = \gamma - \lambda \|\mathbf{e}_i - \mathbf{e}_j\|_2^2$.
- ④ Perte totale : $\mathcal{L} = \text{BCE}(s_{ij}, y_{ij}) + \lambda_{\text{entropy}} (-\sum_f w_f \log w_f)$.
- ⑤ Rétro-propagation et mise à jour des paramètres $\theta_f, \lambda, \gamma$ via Adam ; arrêt anticipé sur l'AUC de validation.

Aspect	Random Walks (Node2Vec)	FastRP (Our Method)
Time Complexity	$O(rwl \times n)$	$O(E \times d \times p)$
Memory	$O(rwl \times n)$	$O(d \times p)$
Preprocessing	Generate walks	Sparse matrix ops
Scalability	Poor (quadratic)	Excellent (linear)

Where

- r = walks per node (typically 10-50)
- w = walk length (typically 80-100)
- l = training epochs (typically 100)
- E = number of edges, d = embedding dimension, p = number of powers

Real Numbers for our dataset

- Node2Vec: $\sim 28K \times 10 \times 80 \times 100 = 2.24B$ operations
- FastRP: $\sim 500K \text{ edges} \times 256 \times 3 = 384M$ operations
- **$\sim 6x$ speedup!**

Pourquoi les embeddings par projection aléatoire sont meilleurs

① Global Structure Preservation:

- Random walks are local (limited by walk length)
- Matrix powers capture global connectivity patterns

② Multi-Scale Relationships:

- Powers 1,2,3 capture immediate, community, and domain-level relationships
- Random walks only see fixed-length neighborhoods

③ Deterministic Features:

- No sampling variance
- Reproducible results
- Better for analysis and debugging

④ Theoretical Guarantees:

- Johnson-Lindenstrauss lemma provides distance preservation bounds
- Random walks have no such guarantees

⑤ Heterogeneous Graph Support:

- Natural handling of different relationship types
- Random walks struggle with heterogeneous graphs

Link Prediction Results

- **Training AUC:** 0.94
- **Validation AUC:** 0.94
- **Training Time:** 15 minutes (vs. hours for random walks)

Importance des caractéristiques apprises

Relationship Type	Weight
Conference relationships (ACA)	70%
Collaboration patterns (AAA)	25%
Topic similarity (ATA)	5%

Impact du filtrage de nœuds

- Original: 28,702 authors
- Filtered (min-degree=5): 27,821 authors
- Removed 881 low-degree "noise" nodes
- Improved clustering quality

Example: Predicting if Alice and Bob will collaborate

Step 1: Alice (ID=1000), Bob (ID=1500)

Step 2: Extract their embeddings

```
z_alice = Z[1000, :] # 256-dimensional vector
```

```
z_bob = Z[1500, :] # 256-dimensional vector
```

Step 3: Compute distance

```
distance_sq = norm(z_alice - z_bob)**2
```

```
# Result: 2.34
```

Step 4: Predict probability

```
gamma = 0.8 # learned intercept
```

```
slope = 0.5 # learned slope
```

```
probability = sigmoid(gamma - slope * 2.34)
```

```
# Result: 0.41
```

Result: 41% chance Alice and Bob will collaborate

How did we get z_alice? From weighted combination of 9 different relationship features!

Projection aléatoire vs méthodes par marches aléatoires

Advantages of Projection aléatoire:

- ✓ **Speed:** 6x faster than Node2Vec
- ✓ **Memory:** Constant memory usage
- ✓ **Quality:** Better global structure capture
- ✓ **Deterministic:** No sampling variance
- ✓ **Scalable:** Linear time complexity
- ✓ **Interpretable:** Feature weights show what matters
- ✓ **Heterogeneous:** Natural multi-relation support

Potential Disadvantages:

- ! **Complexity:** More parameters to tune (α , β , powers)
- ! **Theory gap:** Less established than random walks
- ! **Implementation:** Requires careful sparse matrix handling

Bottom Line

Projection aléatoire provides superior speed-quality tradeoff for heterogeneous graphs

Points à retenir

1. Core Insight

Random projections + matrix powers = efficient graph embeddings

2. Mathematical Foundation

Johnson-Lindenstrauss lemma justifies random projection approach

3. Engineering Innovation

Iterative computation avoids memory explosion: $\mathbf{M} \circ (\mathbf{M} \circ \mathbf{R})$ not $\mathbf{M}^2 \circ \mathbf{R}$

4. Performance Gains

6x speedup with comparable/better quality vs. random walks

5. Practical Impact

Real-world heterogeneous graphs benefit from multi-relation modeling

The Big Picture

Merci !

Code et implémentation

- Available on GitHub: [Your Repository]
- Python implementation with PyTorch
- Supports CPU, CUDA, and Apple Silicon (MPS)

Fichiers clés

- `src/model.py`: Projection aléatoire implementation
- `src/main.py`: Training pipeline
- `scripts/plot_umap.py`: Visualization tools

Essayez vous-même

```
python src/main.py -epochs 30 -meta-paths AAA ACA ATA
```

Questions?

Données d'entrée

- Authors: 28,702
- Conferences: 20
- Terms: 8,920

Matrices de relations

- $\mathbf{A}_{AA}$: $28,702 \times 28,702$ (sparse)
- $\mathbf{A}_{AC}$: $28,702 \times 20$
- $\mathbf{A}_{AT}$: $28,702 \times 8,920$
- $\mathbf{A}_{CA}$: $20 \times 28,702$
- $\mathbf{A}_{TA}$: $8,920 \times 28,702$

Matrices des méta-chemins

- $\mathbf{M}_{AAA}$: $28,702 \times 28,702$
- $\mathbf{M}_{ACA}$: $28,702 \times 28,702$
- $\mathbf{M}_{ATA}$: $28,702 \times 28,702$

Répartition mémoire

- Sparse matrices: $\sim 50\text{MB}$
- Feature tensor: $\sim 2.5\text{GB}$
- Total: $\sim 3\text{GB}$ (vs. $50\text{GB}+$ for naive approach)

Détails clés de l'implémentation

- Used SciPy sparse matrices throughout
- Never materialize full $\mathbf{M}^2$ matrices (would exceed memory)
- Iterative computation: $\mathbf{M} \circ (\mathbf{M} \circ \mathbf{R})$ instead of $\mathbf{M}^2 \circ \mathbf{R}$
- Disk cache for meta-path matrices
- GPU optimization with hybrid CPU/GPU approach

figures/embedding.png